
AdaCtrl: 通过难度感知预算实现自适应与可控推理

黄世珏 ^{α *}, 王泓儒 ^{β *}, 钟万钧*, 苏兆晨 ^{α} , 冯佳展 ^{σ} ,
曹博文 ^{β} , 冯以仁 (May) ^{α †}
 ^{α} 香港科技大学
 ^{β} 香港中文大学, ^{σ} 北京大学

Abstract

现代大型推理模型通过采用复杂的推理策略, 展现出令人印象深刻的问题解决能力。然而, 它们常常难以平衡效率与效果, 经常为简单问题生成不必要的冗长推理链。在这项工作中, 我们提出了 AdaCtrl, 一个新颖的框架, 旨在支持难度感知的自适应推理预算分配以及用户对推理深度的显式控制。AdaCtrl 根据自我评估的问题难度动态调整其推理长度, 同时也允许用户手动控制预算, 以优先考虑效率或效果。这是通过一个两阶段的训练流程实现的: 一个初始的冷启动微调阶段, 用于灌输自我感知难度和调整推理预算的能力; 随后是一个难度感知的强化学习阶段, 该阶段在在线训练过程中根据模型不断演进的能力, 优化其自适应推理策略并校准其难度评估。为了实现直观的用户交互, 我们设计了显式的长度触发标签, 作为预算控制的自然接口。实证结果表明, AdaCtrl 能够根据估计的难度自适应调整推理长度, 与同样结合了微调和强化学习的标准训练基线相比, 在需要精细推理、更具挑战性的 AIME2024 和 AIME2025 数据集上, 它在提升性能的同时, 分别将响应长度减少了 10.06% 和 12.14%; 而在更简洁的响应就足够的 MATH500 和 GSM8K 数据集上, 则分别减少了 62.05% 和 91.04%。此外, AdaCtrl 使用户能够精确控制推理预算, 从而生成满足特定需求的定制化响应。进一步的分析还表明, AdaCtrl 能够准确估计问题难度, 并根据这些评估分配推理预算¹。

1 引言

随着测试时缩放技术的出现 [Snell et al., 2024, Muennighoff et al., 2025, Balachandran et al., 2025, Zhang et al., 2025], 诸如 Deepseek R1 [DeepSeek-AI et al., 2025] 和 OpenAI o1 [OpenAI, 2024] 等大型推理模型通过在生成最终答案前彻底探索多种推理路径, 已在多种任务上展现出卓越性能。虽然这种深入且冗长的推理过程能显著提升模型解决复杂问题的能力, 但它不可避免地会引入显著的推理开销, 并在处理较简单问题时导致不必要的过度思考 [Chen et al., 2025, Sui et al., 2025]。例如, 即使面对像“计算 $\log_2(64)$ ”这样简单直接的问题, 这些模型仍倾向于进行冗长的思维链推理 [Wei et al., 2022], 不必要地运用规划、反思和验证等高级元推理技能 [Ryan et al., 2016, Li et al., 2025, Wang et al., 2025a]。这种行为虽然对复杂查询有益, 却会带来过度的延迟和计算成本, 对用户体验产生负面影响 [Wang et al., 2025c]。

¹此项工作仍在进行中, 代码将在 <https://github.com/JoeYing1019/AdaCtrl> 发布。

Problem: Evaluate $\log_2(64)$.

Answer: 6

User-Controllable Difficulty-Aware Tags [Easy] Efficiency ↑ Effectiveness ↓

Response (252 tokens):

[Easy] To evaluate $\log_2(64)$, we need to determine the power to which the base 2 must be raised to obtain 64. Let's break this down step by step.

1. Identify the relationship: We need to find x such that $2^x = 64$.
2. Express 64 as a power of 2: We know that: $2^1 = 2$, $2^2 = 4$, $2^3 = 8$, $2^4 = 16$, $2^5 = 32$, $2^6 = 64$.
3. Determine the exponent: From the above calculations, we see that $2^6 = 64$.
4. Conclusion: Since $2^6 = 64$, it follows that $\log_2(64) = 6$. Therefore, the final answer is `\boxed{6}`.

User-Controllable Difficulty-Aware Tags [Hard] Efficiency ↓ Effectiveness ↑

Response (1144 tokens):

[Hard] Okay, so I need to evaluate $\log_2(64)$. Hmm, logarithms can sometimes be tricky, but I remember that logarithms are about finding exponents. Let me recall... If I have $\log_b(a) = c$, that means $b^c = a$. So in this case, $\log_2(64) = c$ would mean $2^c = 64$.

Wait, right! $2^6 = 64$. So that means $c = 6$. Therefore, $\log_2(64) = 6$.

But let me double-check to make sure I didn't skip a step. Maybe I should write it out step by step.

Alternatively, if I didn't remember the powers of 2 off the top of my head, maybe I could break it down. Let me see.

Another way to think about it is using logarithm properties. But maybe that's overcomplicating. Thus, the value of $\log_2(64) = 6$ is `\boxed{6}`.

Model-Adaptive Difficulty-Aware Tags [Easy] or [Hard] ? Efficiency ↔ Effectiveness ↔

Response (386 tokens):

[Easy] To evaluate $\log_2(64)$, we need to determine the power to which the base 2 must be raised to obtain 64. Let's break this down step by step.

1. Express 64 as a power of 2: We know that: $2^6 = 64$. This can be verified by calculating: $2^1 = 2$, $2^2 = 4$, $2^3 = 8$, $2^4 = 16$, $2^5 = 32$, $2^6 = 64$.
2. Use the property of logarithms: Since $2^6 = 64$, we can take the logarithm base 2 of both sides: $\log_2(2^6) = \log_2(64)$.
3. Simplify the left-hand side using the power rule of logarithms: The power rule states that $\log_b(a^c) = c \log_b(a)$. Applying this rule, we get: $\log_2(2^6) = 6 \log_2(2)$.
4. Simplify further using the fact that $\log_b(b) = 1$: Since $\log_2(2) = 1$, we have: $6 \log_2(2) = 6 \cdot 1$. Therefore, $\log_2(64) = 6$. The final answer is: `\boxed{6}`.

图 1: 针对同一问题, AdaCtrl 支持三种推理模式: 简单模式以较少的令牌提供简洁答案; 困难模式以较多的令牌提供详尽回答; 自适应模式则根据问题复杂度动态分配推理预算。

近期研究探索了多种方法来提高推理效率并缓解过度思考问题。一些方法旨在通过微调 [Munkhbat et al., 2025, Ma et al., 2025] 或强化学习 [Arora and Zanette, 2025, Aggarwal and Welleck, 2025] 强制简洁性, 从而减少所有问题 (无论其实际复杂度是简单还是困难) 的推理长度。然而, 这种通用压缩策略可能会牺牲真正需要深入分析的复杂任务上的推理质量。另一方面, 一些方法试图精确控制每个问题的令牌预算, 以寻求更高效且准确的推理过程 [Nayab et al., 2025, Muennighoff et al., 2025, Xu et al., 2025]。尽管如此, 这种细粒度控制通常依赖于准确的难度评估和底层大语言模型的卓越能力 (例如精确的指令遵循), 这在实践中可能不够灵活且脆弱。预先确定最优令牌预算本身就具有挑战性, 并且模型可能难以可靠地解释或执行细粒度指令。这些局限性凸显了对一个既能根据任务复杂度进行自适应又易于控制的统一框架的需求。为此, 我们提出了 AdaCtrl, 一个新颖的框架, 旨在: 1) 允许模型根据自我评估的难度动态调整其推理努力 (即自适应推理模式); 以及 2) 使用户能够根据问题难度指定推理模式 (即可控的预算分配)。如图 1 所示, 给定一个查询, 其中两种模式是手动指定的, 允许用户控制推理预算: (i) 简单模式, 优先考虑效率, 对任何问题提供简洁的回应; (ii) 困难模式, 旨在通过详细阐述完整的推理过程并提供详细信息来追求更高的有效性。此外, 自适应模式会根据输入查询的复杂性自动调整推理努力, 在无需人工干预的情况下实现有效性与效率之间的平衡。为实现这一点, 我们首先策划了一个涵盖简单和困难子集的数据集, 并在模型的回应前插入相应的指示标签 (即 “[简单]” 或 “[困难]”)。然后, 我们利用冷启动微调赋予模型基础能力, 以估计问题的复杂性并相应地分配推理预算, 从而实现自我感知的难度估计。此外, 我们采用了难度感知的强化学习框架, 并精心设计了奖励机制, 以校准其对问题难度的自我评估 (即难度估计校准奖励), 并优化模型的自适应推理行为 (即难度感知长度奖励)。

在四个基准测试上的实验结果表明, 它在四个数据集上优于大多数基线, 同时高效地管理推理预算。具体而言, 与标准的 SFT + RL 基线相比, 10.14% 的准确率提升, 同时将回应长度减少了多达 91.04%。进一步的分析表明, 在强化学习过程中实现了准确的难度估计, 并在超参数变化下保持了稳健的性能。总体而言, 贡献如下:

- 我们引入了 AdaCtrl, 一个用于自适应和可控推理的统一框架, 支持效率与性能之间的动态权衡, 允许模型估计问题难度并自行调整推理模式, 同时也允许用户指定所需的推理模式, 以满足实践中多样化的需求。

- 我们提出了一种两阶段训练范式，将冷启动微调和难度感知强化学习相结合，以培养模型对问题难度的自我意识，并支持难度感知的预算分配，同时考虑了模型能力的差异性和动态性。
- 在四个基准数据集上的实证结果表明，。进一步的分析揭示，，能够准确实现难度感知的预算分配。

2 相关工作

通过监督微调提升推理效率。尽管大语言模型通过生成详尽的多步推理链在复杂任务上取得了令人印象深刻的性能 [Dubey et al., 2024, Su et al., 2025]，但这种能力可能导致在处理更简单的查询时产生过度的冗长和计算开销。这种“过度思考”现象推动了关于提升推理效率的研究 [Qu et al., 2025a, Wang et al., 2025c]。一种主要的策略涉及使用监督微调来引导模型进行更简洁的推理。一些 SFT 工作专注于使用本质上更短的推理路径进行训练。例如，模型通过在数据生成过程中使用特定的提示来学习遵守令牌预算 [Han et al., 2024]。其他方法则从最佳 N 采样中提炼简洁路径 [Munkhbat et al., 2025]，或者对模型进行微调，使其在模型已有信心的样本上省略中间步骤 [Yu et al., 2024]。另一个 SFT 方向是压缩现有的推理链。Kang et al. [2024] 使用 GPT-4 [Achiam et al., 2023] 作为压缩器，然后在这些从长到短的思维链映射上微调模型。LMskip [Liu et al., 2024] 在步骤约束下诱导跳步行为。SPIRIT-FT [Cui et al., 2025] 使用困惑度作为指导来识别关键推理步骤以进行剪枝。TokenSkip [Xia et al., 2025] 分析思维链输出中的令牌重要性以实现可控压缩。这些 SFT 方法减少了长度，但通常强制要求一种普遍性的简洁，这可能不适合复杂问题，并且通常缺乏对难度或用户预算控制的自我评估。

通过强化学习提升推理效率。强化学习为优化推理效率提供了另一个重要途径，它建立在成功开发如 DeepSeek-Coder [Guo et al., 2025] 等模型深度推理能力的基础上。许多强化学习方法结合了基于长度的显式奖励，以在保证准确性的同时鼓励简洁性。一些方法将生成长度与任务难度或提示中的指令联系起来：DAST [Shen et al., 2025] 通过奖励塑形使思维链长度适应问题复杂性，而 LCPO [Aggarwal and Welleck, 2025] 使用提示中指定的目标来控制长度。其他技术则相对于基线对长度奖励进行归一化，例如 O1-Pruner [Luo et al., 2025]、Arora and Zanette [2025] 的每提示归一化以及 Kimi 1.5 报告 [Team, 2025] 中的方法。Yeo et al. [2025] 提出了一种余弦奖励来有效管理长度，同时也指出了模型人为延长推理的“长度黑客”问题。除了显式的长度奖励，其他强化学习策略还包括用于测试时优化的元强化学习 [Qu et al., 2025b]、用于预算感知的效用最大化 [Yu et al., 2025b]、结合启发式的偏好优化 [Chen et al., 2025]，以及减轻 GRPO 对更长轨迹的偏好 [Liu et al., 2025]。然而，这些方法通常缺乏 AdaCtrl 的难度感知标签所提供的对推理深度的显式用户控制，并且没有优先训练模型对问题难度的自我认知。我们的两阶段 SFT-RL 框架独特地解决了这些方面，实现了自主和用户影响下的推理预算控制。

3 方法

在本节中，我们将阐述我们提出的 AdaCtrl 的设计，其包括：(1) 为难度估计和难度感知的预算调整提供初始化的冷启动微调；(2) 提升模型在响应长度控制和难度估计方面能力的难度感知强化学习框架。整个框架如图 2 所示。

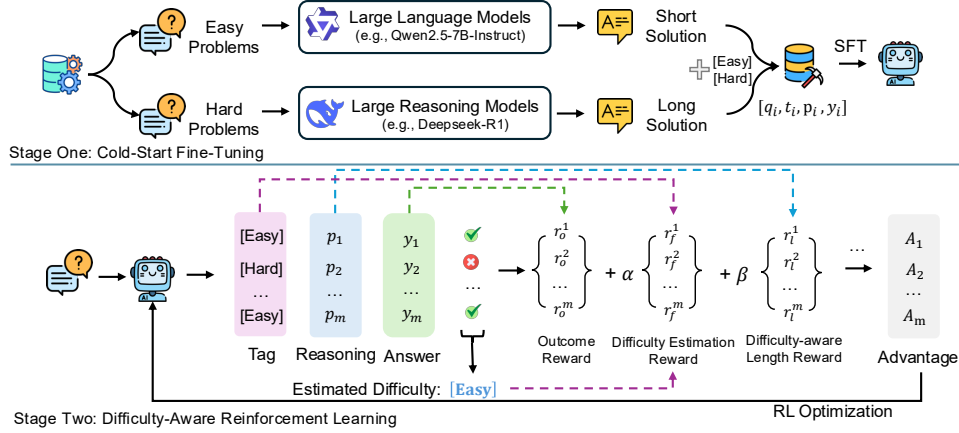


图 2: AdaCtrl 包含一个两阶段训练流程：冷启动微调首先利用短推理轨迹和长推理轨迹来建立基本的预算意识；随后使用一个难度感知的强化学习框架来校准问题难度估计并发展自适应的推理策略。

3.1 冷启动微调

此阶段主要旨在使模型具备遵循包含难度感知标签（例如 “[Easy]” 和 “[Hard]”）的输出格式的能力，并相应地控制响应长度²。为了为此目的策划合适的训练数据，我们直接从 DeepMATH 数据集 [He et al., 2025] 中选择简单和困难的问题，该数据集为每个问题提供了难度标注。对于简单问题 $\{q_1^e, q_2^e, \dots, q_n^e\}$ ，我们利用指令模型 $\mathcal{M}$ 来生成简洁的响应，而对于困难问题 $\{q_1^h, q_2^h, \dots, q_m^h\}$ ，则使用一个强大的大型推理模型 $\mathcal{R}$ 来生成推理轨迹。然后，我们基于答案正确性对两个子集的解进行过滤：

$$\mathcal{D}^e = \{(q_i^e, [p_i, y_i]) | [p_i, y_i] = \mathcal{M}(q_i^e), \mathbb{I}(y_i, \hat{y}_i) = 1\}, \quad (1)$$

$$\mathcal{D}^h = \{(q_i^h, [p_i, y_i]) | [p_i, y_i] = \mathcal{R}(q_i^h), \mathbb{I}(y_i, \hat{y}_i) = 1\}, \quad (2)$$

其中 $[r_i, y_i]$ 是模型生成的响应，包含推理过程 p_i 和预测答案 y_i ， $\hat{y}_i$ 是对应样本的真实答案，而 $\mathbb{I}(y_i, \hat{y}_i) = 1$ 表示模型响应 $[p_i, y_i]$ 在通过答案 $\hat{y}_i$ 验证后是正确的。然后，通过在 $\mathcal{D}^e$ 中的响应前添加 “[Easy]” 标签，在 $\mathcal{D}^h$ 中的响应前添加 “[Hard]” 标签，我们构建了一个用于冷启动训练的数据集 $\mathcal{D} = \{(q_i, [t_i, p_i, y_i])\}$ ，其中 t_i 是难度感知标签。经过冷启动微调后，模型学会了遵循指定的格式，并根据其对每个问题估计的难度生成解决方案，例如为困难问题分配更多的推理令牌，以实现更深入和更多样化的推理路径。值得注意的是，这也为非专业用户提供了更大的可控性，他们可以通过使用简单的标签如 “[Easy]” 和 “[Hard]” 来轻松引导模型的行为。

3.2 难度感知强化学习

冷启动微调后，我们采用组相对策略优化（GRPO）[Shao et al., 2024] 作为强化学习（RL）算法，因为它能产生多次 rollout，这些 rollout 可以自然地用于从训练模型的角度估计给定问题的难度。具体来说，我们精心设计了三种奖励来指导难度感知强化学习优化：

结果准确性奖励。 基于规则的结果准确性奖励已在 RL 训练中被广泛使用 [Guo et al., 2025, Aggarwal and Welleck, 2025, Arora and Zanette, 2025, Yu et al., 2025a, Wang et al., 2025b]，它评

²为了保持控制的简便性和确保可靠性，我们不考虑更细粒度的类别。

估生成响应的正确性:

$$r_o(y_i) = \begin{cases} 1.0 & \mathbb{I}(y_i, \hat{y}_i) = 1 \\ -1.0 & \text{otherwise} \end{cases} \quad (3)$$

为确保更可靠的验证, 我们明确要求模型以指定格式 (即在 `\boxed{\}` 内) 呈现其最终答案。

难度估计校准奖励。 准确估计问题难度是难度感知预算分配的一个基本方面。然而, 由于问题的难度在不同模型之间可能存在显著差异, 并且获取可靠的、模型特定的难度标签也具有挑战性。为了解决这个问题, 我们设计了一个奖励函数, 该函数利用 GRPO 的多次 rollout, 以一种自然有效的方式在在线训练期间估计一个黄金难度。具体来说, 我们使用导致正确答案的 rollout 频率来校准模型的估计难度。如果准确 rollout 的频率超过预定义阈值 δ , 则将问题标记为简单, 否则标记为困难。此外, 在 rollout 过程中, 模型需要在整个响应的开头生成一个正确的难度感知标签以匹配其能力, 因此奖励函数是基于生成的难度感知标签 t_i 与由多次 rollout 确定的估计标签 $\hat{t}_i$ 之间的匹配度来设计的:

$$r_f(y_i) = \begin{cases} 1.0 & \mathbb{I}(t_i, \hat{t}_i) = 1 \\ 0.0 & \mathbb{I}(t_i, \hat{t}_i) = 0 \\ -1.0 & t_i \text{ cannot be found in } y_i \end{cases} \quad (4)$$

难度感知长度奖励。 与之前鼓励模型对所有问题生成简洁响应的相关工作不同 [Arora and Zanette, 2025, Aggarwal and Welleck, 2025], 我们希望仅对简单问题鼓励这种行为, 并保持长思考能力以更好地处理困难问题。这有助于防止过度优化, 即模型在必要时可能无法进行深入思考。为了防止不必要的过度思考, 我们设计了一个难度感知长度奖励, 当生成的难度感知标签 t_i 为 “[Easy]” 时, 鼓励简洁的响应:

$$r_l(y_i) = \begin{cases} 1.0 - \frac{1 - \cos((l_i^j / L_i)\pi)}{2} & t_i = [\text{Easy}] \\ 0.0 & \text{otherwise} \end{cases} \quad (5)$$

其中 l_i^j 是问题 q_i 的第 j 次 rollout 长度, L_i 是问题 q_i 的 rollout 组中的最大长度。我们利用了余弦函数在特定区间 (即 $0-\pi$) 内的单调性。该函数易于调整且行为平滑, 正如最近一项研究所证明的那样 [Yeo et al., 2025]。因此, 难度感知长度奖励会随着生成响应长度的增加而分配较低的分。

总体奖励与目标。 为增强模型的自适应推理能力, 并在在线训练过程中校准其对问题难度的自我评估, 强化学习过程中的总奖励通过整合三个专门设计的奖励来计算:

$$r(y_i) = r_o(y_i) + \alpha \cdot r_f(y_i) + \beta \cdot r_l(y_i) \quad (6)$$

其中 α 和 β 是超参数。在优化过程中, 我们从 DeepMATH [He et al., 2025] 中采样一组涵盖简单和困难问题的题目集合, 以使模型能够学习针对不同类型问题的动态策略。遵循 Yu et al. [2025a], He et al. [2025], 策略模型 π_θ 通过以下目标进行优化:

$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E}_{(q,a) \sim \mathcal{D}, \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|q)} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left(\min \left(\frac{\pi_\theta(o_{i,t} | q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} | q, o_{i,<t})}(\theta) \hat{A}_{i,t}, \right. \right. \right. \\ \left. \left. \left. \text{clip} \left(\frac{\pi_\theta(o_{i,t} | q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} | q, o_{i,<t})}(\theta), 1 - \varepsilon, 1 + \varepsilon \right) \hat{A}_{i,t} \right) - \beta' D_{\text{KL}}(\pi_\theta || \pi_{\text{ref}}) \right) \right], \quad (7)$$

其中 G 是组大小, o_i 是 rollout 响应, $\hat{A}_{i,t}$ 是通过组内奖励进行归一化计算得到的第 i 个响应的优势值, 而 β', ε 是超参数。通过这一强化学习过程, 模型能够更有效地评估问题相对于其自身能力的难度, 并据此发展自适应推理策略。

4 实验

4.1 实验设置

模型与数据集。我们采用 Qwen2.5-7B-Instruct 和 Qwen2.5-14B-Instruct [Qwen et al., 2025] 作为训练的主干模型。在冷启动微调阶段，训练问题来自 DeepMATH 数据集 [He et al., 2025]，该数据集为每个问题分配了从 1 到 9 的难度等级。我们将难度等级为 5 及以下的问题归类为简单问题，难度等级高于 5 的问题归类为困难问题。对于简单问题，主干模型被用来生成简洁的回复。相反，对于困难问题，我们则引入由 Deepseek R1 [DeepSeek-AI et al., 2025] 生成的大量长形式推理轨迹。在使用真实答案过滤这些轨迹后，我们构建了一个包含 8K 个实例的冷启动 SFT 数据集，其中包括 4K 个带有短推理链和 4K 个带有长推理链的实例。为了支持难度感知的强化学习，我们进一步从 DeepMATH 中采样了额外的 30K 个示例，这些示例包含 10K 个简单问题和 20K 个困难问题，并且与冷启动微调数据集中的问题不同。

基线模型。为了评估 AdaCtrl 的有效性，我们将其与几个共享相同主干模型的基线进行比较。这些基线包括：(1) 基础模型：未经修改的基础指令微调模型（即 Qwen2.5-7B-Instruct, Qwen2.5-14B-Instruct）；(2) S1.1：通过在 S1-1.1K 数据集 [Muennighoff et al., 2025] 上进一步训练基础模型得到的推理增强模型；(3) R1-SFT：一个在精选数据集上微调基础模型得到的基线模型，该数据集包含来自冷启动 SFT 数据集的问题以及由 Deepseek R1 生成的所有回复；(4) R1-SFT-RL：一个强化学习模型，在 (3) 的基础上，使用前述 30K RL 数据集上的结果准确率奖励进行训练；(5) Cold-Start：一个在构建的冷启动 SFT 数据集上微调基础模型得到的模型；(6) Cold-Start-RL：一个基于 (5) 的强化学习模型，使用相同 30K RL 数据集上的结果准确率奖励进行训练。

训练细节。对于冷启动微调，我们使用 ms-swift 框架 [Zhao et al., 2024]，学习率为 $1e-5$ ，批大小为 8。对于难度感知强化学习，我们采用 VeRL [Sheng et al., 2025]，所有 RL 实验均在统一设置下进行。具体来说，我们遵循 DAPO [Yu et al., 2025a] 来消除 KL 散度。策略模型使用 AdamW 优化器进行优化，学习率为 $1e-6$ ，批大小为 256，微批大小为 32。并且我们在训练期间将 α 和 β 的值均设为 0.5， δ 的值设为 0.625。在 rollout 阶段，每个提示采样 16 个回复，最大生成长度设置为 24K 个 token。所有实验均在 NVIDIA H800 GPU 上进行。

基线提示

请逐步推理以回答以下数学问题，并将你的最终答案以 `\boxed{答案}` 的格式呈现。

难度感知提示

回答以下数学问题，判断所给问题的难度（简单/困难）并以格式：[此处为难度] 开始你的回答，并将你的最终答案以格式 `\boxed{答案}` 呈现。

评估设置。我们在四个数学数据集上评估我们的方法，这些数据集涵盖了从简单到具有挑战性的问题：AIME2024 [Art of Problem Solving, n.d.]、AIME2025 [Art of Problem Solving, n.d.]、MATH500 [Hendrycks et al., 2021] 和 GSM8K [Cobbe et al., 2021]。前两个数据集包含更具挑战性的数学奥林匹克风格问题，而后两个数据集主要包含较简单的、小学水平的问题，其中 GSM8K 是最简单的。由于 AIME2024 和 AIME2025 各自仅包含 30 个样本，我们对这两个数据集报告了 8 次独立运行的平均性能。所有评估均使用一致的推理超参数进行，温度设置为 0.7，top-p 值设置为 0.8。在本文中，我们使用一个基线提示来评估所有未融入难度感

表 1: AdaCtrl 自适应模式的主要结果。我们计算了两个指标：Acc.(%) 是平均准确率，Len. 是平均生成的令牌数。

模型	AIME2024		AIME2025		MATH500		GSM8K	
	Acc.(%)↑	Len. ↓	Acc.(%)↑	Len. ↓	Acc.(%)↑	Len. ↓	Acc.(%)↑	Len. ↓
<i>Qwen2.5-7B-Instruct</i>								
Qwen2.5-7B-Instruct	11.25	1805.60	7.08	1174.06	73.00	628.91	91.58	272.93
S1.1-7B	16.67	19022.60	18.33	18302.72	68.80	5824.47	91.89	1838.12
R1-SFT-7B	12.08	20767.36	13.33	20184.36	62.00	8103.41	87.34	3336.81
Cold-Start-SFT-7B	11.25	5553.00	7.92	6237.49	71.00	805.78	90.22	345.44
R1-SFT-RL-7B	21.25	18778.92	17.50	17924.35	66.80	8421.57	88.93	3896.76
Cold-Start-RL-7B	18.33	16911.63	14.58	15941.76	73.00	3797.60	90.67	369.94
AdaCtrl-7B	21.25	16889.50	19.17	15749.08	74.00	3195.69	90.98	349.34
<i>Qwen2.5-14B-Instruct</i>								
Qwen2.5-14B-Instruct	11.67	1043.20	10.42	1136.26	73.60	568.28	93.86	215.42
S1.1-14B	32.92	17278.03	29.58	16167.50	76.00	4393.17	95.15	1460.26
R1-SFT-14B	19.58	20485.99	20.42	18447.68	65.80	6473.98	90.67	2883.76
Cold-Start-SFT-14B	12.92	4734.60	12.50	6431.17	76.20	929.48	93.63	287.72
R1-SFT-RL-14B	24.17	18549.26	23.33	18294.04	70.60	6397.75	91.05	2814.69
Cold-Start-RL-14B	27.50	17722.25	24.17	17088.44	75.60	4657.38	94.01	378.39
AdaCtrl-14B	34.58	15173.21	25.83	14476.93	79.20	3209.84	94.09	316.12

知推理的基线方法。此外，我们设计了一个简单的难度感知提示，该提示源自基线提示，以支持我们提出的方法，具体细节见框 1（基线提示）和框 2（难度感知提示）。为了量化模型的有效性和效率，我们报告了两个指标：准确率（Acc.）和响应中生成的令牌数量（Len.）。

4.2 主要结果

主要结果展示于表 1。我们可以得出以下观察：

AdaCtrl 有效平衡性能与推理预算。 AdaCtrl 展现出具有竞争力的整体性能，在四个数据集上优于大多数基线，同时高效地管理推理预算。具体而言，与基于 Qwen2.5-7B-Instruct 的 RL 基线 R1-SFT-RL 相比，AdaCtrl-7B 在 AIME2024 上取得了相当的准确率，并在 AIME2025、MATH500 和 GSM8K 上分别将准确率进一步提升了 1.67%、7.20% 和 2.05%。同时，它在这些数据集上的响应长度分别大幅减少了 10.06%、12.14%、62.05% 和 91.04%。在 14B 设置下，AdaCtrl-14B 在 AIME2024、AIME2025、MATH500 和 GSM8K 上的准确率分别提高了 10.41%、2.5%、8.6% 和 3.04%，同时在相应数据集上的推理预算压缩了 18.20%、20.92%、49.83% 和 88.77%。与 S1.1-7B 和 S1.1-14B 基线进行比较时，也观察到了类似的趋势。总体而言，这些结果表明 AdaCtrl 能够根据问题难度自适应地分配推理预算，从而有效平衡推理效率与效果。

冷启动微调为自适应预算分配提供了有效基础。 与仅使用 Deepseek R1 的长形式推理轨迹进行训练的 R1-SFT 不同，我们的冷启动微调策略（即 Cold-Start-SFT）结合了简洁和扩展的推理轨迹。这种多样化的训练方案使模型能够习得更高效的推理策略，从而在 7B 模型上，对 AIME2024、AIME2025、MATH500 和 GSM8K 的响应长度分别大幅减少了 73.26%、69.10%、90.06% 和 89.65%，在 14B 模型上分别减少了 76.88%、54.14%、85.64% 和 90.02%。此外，当从冷启动检查点初始化的模型进行额外的强化学习（即 Cold-Start-RL）时，与从 R1-SFT 初始化的模型（即 R1-SFT-RL）相比，它们展现出更优的预算控制能力，在 7B 模型上，对相同数据集的响应长度额外减少了 9.94%、11.06%、54.91% 和 91.04%，在 14B 模型上额外减

表 2: AdaCtrl 中自适应与受控推理模式的比较。简易模式减少了推理令牌数量，但以性能下降为代价，适用于需要快速响应的场景。困难模式使用更多令牌并产生更好的结果。自适应模式自动估计问题难度，以平衡效果与效率。

模型	AIME2024		AIME2025		MATH500		GSM8K	
	准确率 (%)↑	长度↓	准确率 (%)↑	长度↓	准确率 (%)↑	长度↓	准确率 (%)↑	长度↓
AdaCtrl-7B (自适应)	21.25	16889.50	19.17	15749.08	74.00	3195.69	90.98	349.34
AdaCtrl-7B (简易)	14.58	1652.42	10.00	896.14	70.80	652.76	90.75	314.49
AdaCtrl-7B (困难)	21.67	17562.94	22.08	16114.87	71.20	5960.15	92.57	2058.13
AdaCtrl-14B (自适应)	34.58	15173.21	25.83	14476.93	79.20	3209.84	94.09	316.12
AdaCtrl-14B (简易)	15.83	1394.78	11.25	985.35	75.40	578.37	94.31	284.83
AdaCtrl-14B (困难)	36.25	15004.55	26.25	14564.21	74.40	5321.25	93.33	2598.28

表 3: 超参数分析。 α 控制难度估计校准的奖励权重，而 β 对应于难度感知长度目标的奖励权重。参数 δ 作为区分简单与困难样本的预定义阈值。

AdaCtrl-7B	AIME2024		AIME2025		MATH500		GSM8K	
	准确率 (%)↑	长度↓	准确率 (%)↑	长度↓	准确率 (%)↑	长度↓	准确率 (%)↑	长度↓
$\alpha = 0.5, \beta = 0.5, \delta = 0.625$	21.25	16889.50	19.17	15749.08	74.00	3195.69	90.98	349.34
$\alpha = 1.0, \beta = 1.0, \delta = 0.625$	23.75	14762.78	20.00	13986.64	73.80	3134.37	90.30	325.04
$\alpha = 0.5, \beta = 1.0, \delta = 0.625$	22.08	15616.19	18.75	16116.17	73.80	2836.52	91.13	324.10
$\alpha = 1.0, \beta = 0.5, \delta = 0.625$	21.67	17843.66	22.08	16684.28	73.40	3727.77	91.66	365.03
$\alpha = 0.5, \beta = 0.5, \delta = 0.5$	22.92	13790.53	20.42	14040.20	74.00	2888.39	90.60	388.10
$\alpha = 0.5, \beta = 0.5, \delta = 0.375$	20.00	13385.01	20.00	13689.74	74.60	2181.96	90.60	323.27

少了 4.46%、6.61%、27.20% 和 86.56%。这些结果凸显了冷启动微调阶段在为下游任务建立有效的自适应预算分配坚实基础方面的关键作用。

我们的奖励设计同时增强了推理效果与效率。在冷启动微调模型的基础上，AdaCtrl 优于仅基于结果准确性应用强化学习的 Cold-Start-RL，在 7B 模型上，于 AIME2024、AIME2025、MATH500 和 GSM8K 分别实现了 2.92%、4.59%、1.00% 和 0.31% 的显著准确率提升，在 14B 模型上分别提升了 7.08%、1.66%、3.6% 和 0.08%，同时消耗的令牌数少得多。这些准确率的提升归因于我们的奖励函数设计，该设计结合了难度估计校准和难度感知的长度调整。通过引入这些额外的奖励信号，我们的方法使模型能够更准确地评估问题复杂性并相应地分配计算资源，从而在推理效果与效率之间实现了更精细的平衡。

AdaCtrl 的人机协同可控性。 AdaCtrl 引入了难度感知标签，作为生成过程中的先决条件标记，从而实现对响应长度的显式控制。为了评估该机制的可控性，我们通过指定 “[Easy]” 或 “[Hard]” 标签来模拟用户意图，这两个标签分别对应简化推理模式和复杂推理模式。如表 2 所示，实验结果表明我们的方法能够有效控制推理预算。与模型自主判断问题难度的自适应推理模式相比，强制使用 “[Easy]” 模式在所有四个数据集上均导致性能下降。然而，在更具挑战性的 AIME2024 和 AIME2025 数据集上，7B 模型的响应长度也分别大幅减少了 90.22% 和 94.31%，14B 模型则分别减少了 90.81% 和 93.19%。相反，“[Hard]” 模式在大多数数据集上提升了性能，并显著增加了响应长度，在 7B 设置下，较简单的 MATH500 和 GSM8K 数据集上的响应长度分别增长了 86.51% 和 489.15%，在 14B 设置下则分别增长了 65.78% 和 721.93%。这些发现表明，AdaCtrl 能够实现精确的、人机协同的推理预算控制。

表 4: 基于 7B 模型的消融研究。 r_f 为难度估计校准奖励, r_l 为难度感知长度奖励。

准确率 (%)	AIME2024	AIME2025	MATH500	GSM8K
AdaCtrl-7B	21.25	19.17	74.00	90.98
无 r_f	17.08 (-4.17%)	16.25 (-2.92%)	72.20 (-1.80%)	89.92 (1.06%)
无 r_l	15.42 (-5.83%)	16.67 (-2.50%)	68.60 (-5.4%)	90.60 (-0.38%)

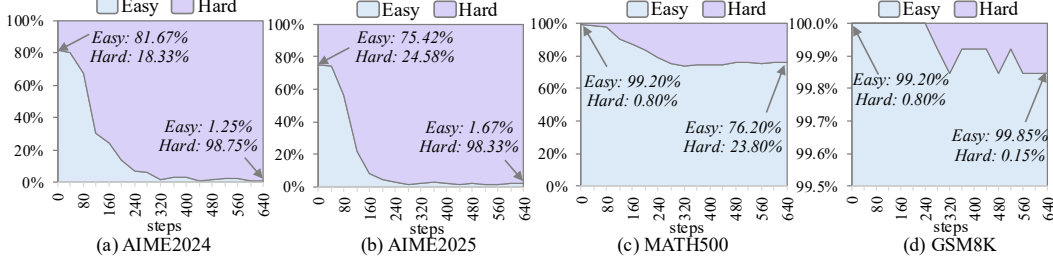


图 3: 强化学习过程中, 不同数据集上难度感知标签的比例动态变化。

4.3 分析

超参数分析。 为了评估我们提出的方法的鲁棒性, 我们进行了两组超参数分析, 总结于表 3。具体来说, 我们考察了: (1) 权重 α 和 β , 它们分别对应难度估计校准奖励和难度感知长度奖励; (2) 难度阈值 δ 。对于奖励权重分析, 我们研究了不同组合 $\{1:0.5:0.5, 1:1:1, 1:0.5:1, 1:1:0.5\}$ 下改变奖励权重的影响。我们观察到准确率基本保持稳定, 而输出长度略有变化。例如, 在 AIME2024 上, 准确率保持在 21-23% 之间, 而响应长度范围在 14K 到 17K 个标记之间。类似地, 在 GSM8K 上, 准确率始终在 90-91% 之间, 而响应长度则在 325-365 个标记之间变化。对于难度阈值分析, 我们进一步将难度阈值 δ 在 $\{0.625, 0.5, 0.375\}$ 范围内进行扫描。较小的 δ 值会将更多样本归类为简单, 从而在保持相当准确率的同时缩短输出长度。例如, 在 MATH500 上, 响应长度从 3195.69 个标记减少到 2888.39 个, 再到 2181.96 个标记, 而准确率保持在 74% 左右。这些结果共同证明了我们提出的方法在不同超参数设置下的鲁棒性。

消融研究。 为了进一步评估所提出的奖励函数的有效性, 我们通过单独移除难度估计校准奖励 r_f 和难度感知长度奖励 r_l 进行了消融研究。如表 4 所示, 移除任一奖励都会导致所有四个数据集的性能出现明显下降。这些结果表明, 结合使用所有设计的奖励对于实现更好的优化至关重要。

AdaCtrl 作为良好的难度评估器。 为了更好地评估 AdaCtrl 的难度估计能力, 我们分析了 AdaCtrl-7B 在强化学习 (RL) 过程中, 在四个数据集上生成难度感知标签的比例。如图 3 所示, 我们首先观察到, 在初始阶段, 模型倾向于将所有数据集中的大多数样本归类为简单。这很可能是由于冷启动的监督微调 (SFT) 主要教导模型以期望的格式显式生成自我感知的难度标签, 而不是从模型自身的角度准确评估问题难度。然而, 在我们设计的难度感知强化学习之后, AdaCtrl 主要将 AIME2024 和 AIME2025 数据集中的大多数问题归类为困难。这些数据集包含具有挑战性的数学奥林匹克级别问题。相比之下, 在 MATH500 数据集中 (该数据集包含简单和困难问题的混合, 其中大多数问题对于当前的大型语言模型来说相对可解), 模型将 76.2% 的问题识别为简单。对于 GSM8K 这个四个数据集中最简单的一个, 超过 99% 的问题被归类为简单, 这也解释了表 2 中报告的 AdaCtrl-14B (简单模式) 在 GSM8K 上的优

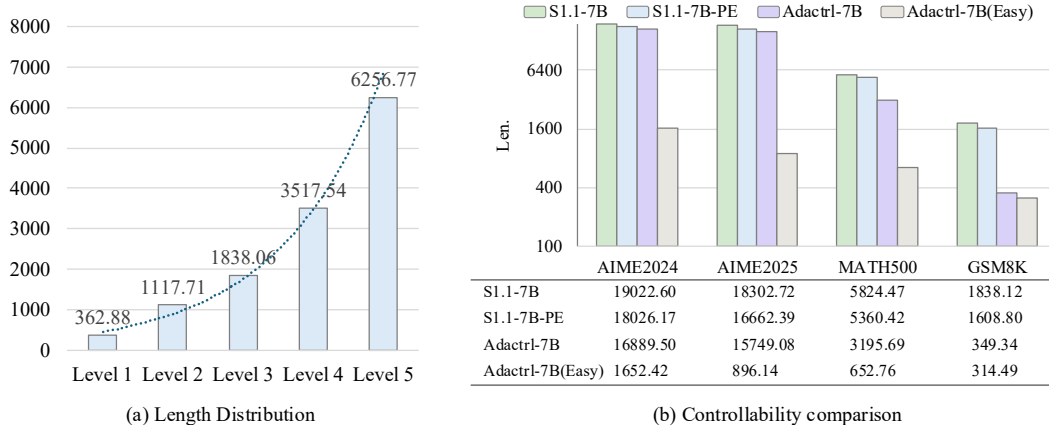


图 4: (a) MATH500 中不同难度级别问题的响应长度，其中级别越高表示问题越具挑战性；(b) AdaCtrl 与基于提示的方法的可控性比较。

异性能。这些结果与数据集的真实难度水平相符，并证明 AdaCtrl 通过强化学习发展出了强大的问题难度估计能力。

AdaCtrl 促进准确的难度感知预算分配。 为了进一步研究 AdaCtrl 的自适应难度感知预算分配能力，我们分析了 AdaCtrl-7B 在 MATH500 数据集上的响应，该数据集为每个问题提供了难度级别标注。如图 4 (a) 所示，随着难度级别从 1 增加到 5，AdaCtrl 生成的响应逐渐变长且更加详细，范围从大约 0.3K 到 6K 个词元。这一趋势表明，AdaCtrl 能够根据其自我评估的问题难度准确调节其推理预算，从而实现计算资源的自动和自适应分配。

可控性分析。 先前的方法通常依赖于普通的提示方法来实现对推理预算的词元级控制。为了评估 AdaCtrl 的可控性，我们将其与一个基于提示的基线进行比较，该基线使用 S1.1-7B 模型作为主干，并遵循 Nayab et al. [2025] 的方法，在用户提示中增加指令：“将答案长度限制在 500 个词元以内。” AdaCtrl 在简单模式和自适应模式下生成的响应长度，以及 S1.1-7B 及其基于提示的预算控制变体生成的响应长度，均在图 4 (b) 中报告。我们可以观察到，通过基于提示的方法（即 S1.1-7B-PE）明确将输出限制在 500 个词元，其在 AIME2024、AIME2025、MATH500 和 GSM8K 数据集上仅分别实现了 5.25%，8.96%，7.97% 和 12.48% 的响应长度缩减，远未达到目标 500 词元的限制。当长度约束设置为 1000 个词元时，我们也观察到了类似的趋势。这些结果表明，由于高度依赖于模型的指令遵循能力，实现对输出长度的精确、细粒度控制具有挑战性，并且当前模型可能难以可靠地解释和执行如此细粒度的提示。相比之下，AdaCtrl 的简单模式实现了对推理预算更大幅度的压缩，在相同数据集上分别将响应长度减少了 90.22%，94.32%，79.57% 和 9.98%。这展示了其优越的可控性，这主要归因于 AdaCtrl 的混合微调策略及其难度感知长度奖励的设计。

生成响应的长度分布。 为了进一步证明 AdaCtrl 的预算分配能力，我们分析了 AdaCtrl-7B 在所有评估基准推理结果中的响应长度分布。具体而言，我们根据模型预测的难度标签，将样本分为简单和困难两类，并分别计算其分布。如图 5 所示，AdaCtrl 清晰地区分了简单和困难问题。值得注意的是，简

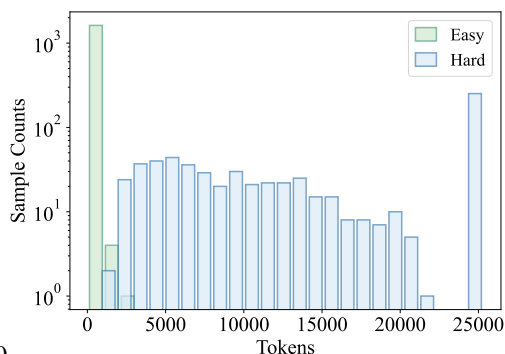


图 5: AdaCtrl-7B 生成响应的长度分布。

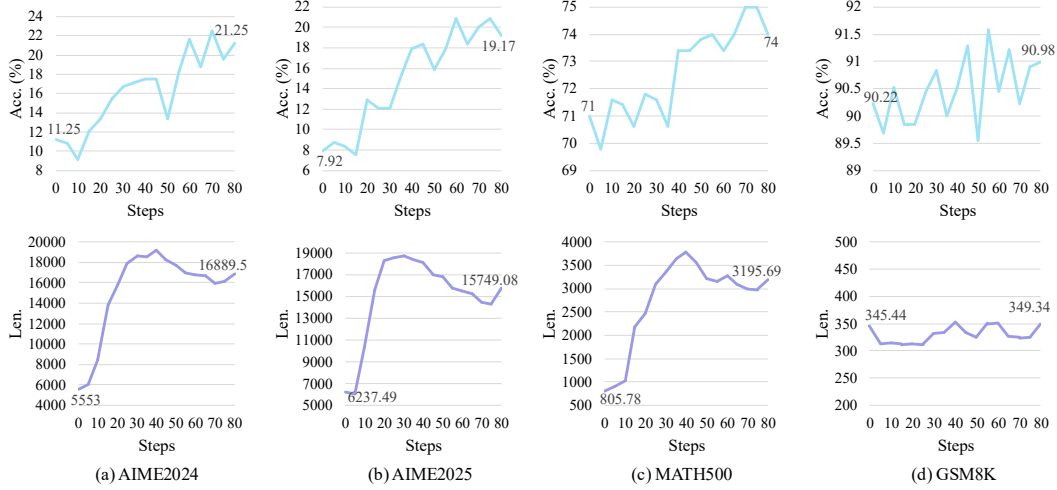


图 6: 强化学习过程中, 不同数据集上准确率和响应长度的训练动态。

单问题的响应长度集中在一个相对较窄的范围内。这些发现表明, 我们的方法在自我评估的难度感知标签指导下, 实现了有效且准确的预算控制。

强化学习期间的训练动态。 为了进一步研究模型如何学习分配自适应推理预算, 我们分析了 AdaCtrl-7B 在整个强化学习训练过程中的性能趋势和预算动态。如图 6 所示, 模型在所有四个数据集上的性能均呈现上升趋势, 表明其推理能力逐步增强。关于预算动态, 我们在不同数据集上观察到不同的模式。在 AIME2024、AIME2025 和 MATH500 上, 平均响应长度在训练早期快速增加, 随后逐渐下降并稳定在比 RL 训练前更长的水平。而对于 GSM8K, 响应长度保持相对稳定, 且接近 RL 训练前观察到的水平。

这些发现表明, 在冷启动微调阶段学习到的推理预算分配对于更复杂的问题 (例如 AIME2024、AIME2025 和 MATH500 中的问题) 是不够的。因此, 模型在 RL 阶段会根据实际问题的难度动态调整其预算。相比之下, 对于相对简单的 GSM8K 数据集, 模型在冷启动微调后已经能够有效地分配最小预算, 这表明它能够区分并处理较简单的问题, 而无需进行重大调整。

5 结论

在本工作中, 我们提出了一个自适应且可控的推理框架, 旨在缓解过度思考问题, 同时赋予用户对计算资源的显式控制。为此, 我们引入了 AdaCtrl, 它同时支持动态推理预算分配和用户导向的预算调整。我们的方法采用了一个两阶段训练流程, 结合了冷启动微调 and 难度感知的强化学习。在四个基准数据集上进行的实验表明, AdaCtrl 能够基于自评估的问题难度有效分配推理预算, 从而在将响应长度动态减少 10%-90% 的同时提升性能。这使得效率与性能之间能够实现灵活的权衡。此外, AdaCtrl 解锁了根据定制需求实现人机协同控制推理预算的潜力。

参考文献

- Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- Pranjal Aggarwal and Sean Welleck. L1: Controlling how long a reasoning model thinks with reinforcement learning, 2025. URL <https://arxiv.org/abs/2503.04697>.
- Daman Arora and Andrea Zanette. Training language models to reason efficiently, 2025. URL <https://arxiv.org/abs/2502.04463>.
- Art of Problem Solving. Aime problems and solutions. https://artofproblemsolving.com/wiki/index.php/AIME_Problems_and_Solutions, n.d. Accessed: 2025-05-22.
- Vidhisha Balachandran, Jingya Chen, Lingjiao Chen, Shivam Garg, Neel Joshi, Yash Lara, John Langford, Besmira Nushi, Vibhav Vineet, Yue Wu, and Safoora Yousefi. Inference-time scaling for complex tasks: Where we stand and what lies ahead, 2025. URL <https://arxiv.org/abs/2504.00294>.
- Xingyu Chen, Jiahao Xu, Tian Liang, Zhiwei He, Jianhui Pang, Dian Yu, Linfeng Song, Qiuzhi Liu, Mengfei Zhou, Zhuosheng Zhang, Rui Wang, Zhaopeng Tu, Haitao Mi, and Dong Yu. Do not think that much for $2+3=?$ on the overthinking of ol-like llms, 2025. URL <https://arxiv.org/abs/2412.21187>.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021. URL <https://arxiv.org/abs/2110.14168>.
- Yingqian Cui, Pengfei He, Jingying Zeng, Hui Liu, Xianfeng Tang, Zhenwei Dai, Yan Han, Chen Luo, Jing Huang, Zhen Li, et al. Stepwise perplexity-guided refinement for efficient chain-of-thought reasoning in large language models. *arXiv preprint arXiv:2502.13260*, 2025.
- DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jiawei Wang, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Shengfeng Ye, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wanjia Zhao, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong

- Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanhong Xu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025. URL <https://arxiv.org/abs/2501.12948>.
- Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Tingxu Han, Chunrong Fang, Shiyu Zhao, Shiqing Ma, Zhenyu Chen, and Zhenting Wang. Token-budget-aware llm reasoning. *arXiv preprint arXiv:2412.18547*, 2024.
- Zhiwei He, Tian Liang, Jiahao Xu, Qiuzhi Liu, Xingyu Chen, Yue Wang, Linfeng Song, Dian Yu, Zhenwen Liang, Wenxuan Wang, Zhuosheng Zhang, Rui Wang, Zhaopeng Tu, Haitao Mi, and Dong Yu. Deepmath-103k: A large-scale, challenging, decontaminated, and verifiable mathematical dataset for advancing reasoning, 2025. URL <https://arxiv.org/abs/2504.11456>.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset, 2021. URL <https://arxiv.org/abs/2103.03874>.
- Yu Kang, Xianghui Sun, Liangyu Chen, and Wei Zou. C3ot: Generating shorter chain-of-thought without compromising effectiveness. *arXiv preprint arXiv:2412.11664*, 2024.
- Zhong-Zhi Li, Duzhen Zhang, Ming-Liang Zhang, Jiaxin Zhang, Zengyan Liu, Yuxuan Yao, Haotian Xu, Junhao Zheng, Pei-Jie Wang, Xiuyi Chen, Yingying Zhang, Fei Yin, Jiahua Dong, Zhiwei Li, Bao-Long Bi, Ling-Rui Mei, Junfeng Fang, Zhijiang Guo, Le Song, and Cheng-Lin Liu. From system 1 to system 2: A survey of reasoning large language models, 2025. URL <https://arxiv.org/abs/2502.17419>.
- Tengxiao Liu, Qipeng Guo, Xiangkun Hu, Cheng Jiayang, Yue Zhang, Xipeng Qiu, and Zheng Zhang. Can language models learn to skip steps? *arXiv preprint arXiv:2411.01855*, 2024.
- Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding r1-zero-like training: A critical perspective. <https://github.com/sail-sg/understand-r1-zero>, 2025.

- Haotian Luo, Li Shen, Haiying He, Yibo Wang, Shiwei Liu, Wei Li, Naiqiang Tan, Xiaochun Cao, and Dacheng Tao. O1-pruner: Length-harmonizing fine-tuning for o1-like reasoning pruning. *arXiv preprint arXiv:2501.12570*, 2025.
- Xinyin Ma, Guangnian Wan, Runpeng Yu, Gongfan Fang, and Xinchao Wang. Cot-valve: Length-compressible chain-of-thought tuning, 2025. URL <https://arxiv.org/abs/2502.09601>.
- Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. s1: Simple test-time scaling, 2025. URL <https://arxiv.org/abs/2501.19393>.
- Tergel Munkhbat, Namgyu Ho, Seohyun Kim, Yongjin Yang, Yujin Kim, and Se-Young Yun. Self-training elicits concise reasoning in large language models, 2025. URL <https://arxiv.org/abs/2502.20122>.
- Sania Nayab, Giulio Rossolini, Marco Simoni, Andrea Saracino, Giorgio Buttazzo, Nicolamaria Manes, and Fabrizio Giacomelli. Concise thoughts: Impact of output length on llm reasoning and cost, 2025. URL <https://arxiv.org/abs/2407.19825>.
- OpenAI. Introducing openai o1, September 2024. URL <https://openai.com/o1/>. Accessed: 2025-05-21.
- Xiaoye Qu, Yafu Li, Zhaochen Su, Weigao Sun, Jianhao Yan, Dongrui Liu, Ganqu Cui, Daizong Liu, Shuxian Liang, Junxian He, et al. A survey of efficient reasoning for large reasoning models: Language, multimodality, and beyond. *arXiv preprint arXiv:2503.21614*, 2025a.
- Yuxiao Qu, Matthew Y. R. Yang, Amrith Setlur, Lewis Tunstall, Edward Emanuel Beeching, Ruslan Salakhutdinov, and Aviral Kumar. Optimizing test-time compute via meta reinforcement fine-tuning, 2025b. URL <https://arxiv.org/abs/2503.07572>.
- Qwen, :, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report, 2025. URL <https://arxiv.org/abs/2412.15115>.
- Tony Ryan, Shu Wen Tay, P. D. Ryan, and C. Anthony Ryan. Systems 1 and 2 thinking processes and cognitive reflection testing in medical students. *Canadian Medical Education Journal*, 7:e97–e103, 2016. URL <https://api.semanticscholar.org/CorpusID:11436274>.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024. URL <https://arxiv.org/abs/2402.03300>.
- Yi Shen, Jian Zhang, Jieyun Huang, Shuming Shi, Wenjing Zhang, Jiangze Yan, Ning Wang, Kai Wang, and Shiguo Lian. Dast: Difficulty-adaptive slow-thinking for large reasoning models, 2025. URL <https://arxiv.org/abs/2503.04472>.

- Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, EuroSys ' 25, page 1279–1297. ACM, March 2025. doi: 10.1145/3689031.3696075. URL <http://dx.doi.org/10.1145/3689031.3696075>.
- Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling llm test-time compute optimally can be more effective than scaling model parameters, 2024. URL <https://arxiv.org/abs/2408.03314>.
- Zhaochen Su, Linjie Li, Mingyang Song, Yunzhuo Hao, Zhengyuan Yang, Jun Zhang, Guanjie Chen, Jiawei Gu, Juntao Li, Xiaoye Qu, et al. Openthinking: Learning to think with images via visual tool reinforcement learning. *arXiv preprint arXiv:2505.08617*, 2025.
- Yang Sui, Yu-Neng Chuang, Guanchu Wang, Jiamu Zhang, Tianyi Zhang, Jiayi Yuan, Hongyi Liu, Andrew Wen, Shaochen Zhong, Hanjie Chen, and Xia Hu. Stop overthinking: A survey on efficient reasoning for large language models, 2025. URL <https://arxiv.org/abs/2503.16419>.
- Kimi Team. Kimi k1.5: Scaling reinforcement learning with llms, 2025. URL <https://arxiv.org/abs/2501.12599>.
- Hongru Wang, Deng Cai, Wanjuan Zhong, Shijue Huang, Jeff Z. Pan, Zeming Liu, and Kam-Fai Wong. Self-reasoning language models: Unfold hidden reasoning chains with few reasoning catalyst. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Findings of the Association for Computational Linguistics: ACL 2025*, pages 5578–5596, Vienna, Austria, July 2025a. Association for Computational Linguistics. ISBN 979-8-89176-256-5. URL <https://aclanthology.org/2025.findings-acl.291/>.
- Hongru Wang, Cheng Qian, Wanjuan Zhong, Xiusi Chen, Jiahao Qiu, Shijue Huang, Bowen Jin, Mengdi Wang, Kam-Fai Wong, and Heng Ji. Acting less is reasoning more! teaching model to act efficiently, 2025b. URL <https://arxiv.org/abs/2504.14870>.
- Rui Wang, Hongru Wang, Boyang Xue, Jianhui Pang, Shudong Liu, Yi Chen, Jiahao Qiu, Derek Fai Wong, Heng Ji, and Kam-Fai Wong. Harnessing the reasoning economy: A survey of efficient reasoning for large language models. *arXiv preprint arXiv:2503.24377*, 2025c.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- Heming Xia, Yongqi Li, Chak Tou Leong, Wenjie Wang, and Wenjie Li. Tokenskip: Controllable chain-of-thought compression in llms. *arXiv preprint arXiv:2502.12067*, 2025.
- Silei Xu, Wenhao Xie, Lingxiao Zhao, and Pengcheng He. Chain of draft: Thinking faster by writing less, 2025. URL <https://arxiv.org/abs/2502.18600>.
- Edward Yeo, Yuxuan Tong, Morry Niu, Graham Neubig, and Xiang Yue. Demystifying long chain-of-thought reasoning in llms, 2025. URL <https://arxiv.org/abs/2502.03373>.
- Ping Yu, Jing Xu, Jason Weston, and Ilia Kulikov. Distilling system 2 into system 1, 2024. URL <https://arxiv.org/abs/2407.06023>.

- Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiase Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Mu Qiao, Yonghui Wu, and Mingxuan Wang. Dapo: An open-source llm reinforcement learning system at scale, 2025a. URL <https://arxiv.org/abs/2503.14476>.
- Zishun Yu, Tengyu Xu, Di Jin, Karthik Abinav Sankararaman, Yun He, Wenxuan Zhou, Zhouhao Zeng, Eryk Helenowski, Chen Zhu, Sinong Wang, et al. Think smarter not harder: Adaptive reasoning with inference aware optimization. *arXiv preprint arXiv:2501.17974*, 2025b.
- Qiyuan Zhang, Fuyuan Lyu, Zexu Sun, Lei Wang, Weixu Zhang, Wenyue Hua, Haolun Wu, Zhihan Guo, Yufei Wang, Niklas Muennighoff, Irwin King, Xue Liu, and Chen Ma. A survey on test-time scaling in large language models: What, how, where, and how well?, 2025. URL <https://arxiv.org/abs/2503.24235>.
- Yuze Zhao, Jintao Huang, Jinghan Hu, Xingjun Wang, Yunlin Mao, Daoze Zhang, Zeyinzi Jiang, Zhikai Wu, Baole Ai, Ang Wang, Wenmeng Zhou, and Yingda Chen. Swift:a scalable lightweight infrastructure for fine-tuning, 2024. URL <https://arxiv.org/abs/2408.05517>.